

Mini-proiecte — lotul 2

Patru mini-proiecte peste **lectia 02**: vizibilitate, contoare atomice, compare-and-set. Teoria e in celalalt repo, `src/teorie/L02-*`.

Regulile raman cele din `GHID.pdf`: cerinta nu-ti spune ce clasa din Java sa folosesti, fiecare program isi tipareste singur verdictul, fiecare program se inchide singur, un mini-proiect = un folder = o clasa.

#	Mini-proiect	Trece prin	Ce trebuie sa iasa
M06	Buton de oprire	L02-A1 L02-A2	A nu se opreste, B se opreste sub 10 ms
M07	Contor de clickuri	L02-A3 L02-B1	A pierde, B exact 400.000
M08	Distributor de bonuri	L02-B2	bonuri 1..100, ecranul arata 100
M09	Licitatie cu o singura oferta	L02-B3 L02-B4 L02-C1	A gresit, B 20/20 corect

M06 — Buton de oprire (2 stagii)

Un robot de curatenie numara rotatiile periei, cat de repede poate, pana cand cineva apasa butonul de oprire. In bucla robotul face **un singur lucru**: numara. Nu tipareste nimic si nu doarme.

`main` porneste robotul, asteapta 500 ms, apasa butonul si asteapta cel mult 2 secunde ca robotul sa se opreasca.

Stagiul A: butonul e un camp obisnuit. Stagiul B: acelasi cod, dar butonul e declarat astfel incat robotul sa vada apasarea imediat. **Diferenta dintre A si B trebuie sa fie de un singur cuvant in cod.**

Fiecare stagiou afiseaza daca robotul s-a oprit, dupa cate milisecunde de la apasare si cate rotatii a facut.

Criteriu de acceptare. La stagiul A robotul **nu** se opreste in cele 2 secunde, iar programul o spune singur (**robotul NU s-a oprit**) si trece mai departe la B. La stagiul B robotul se opreste in mai putin de 10 ms de la apasare. Programul se termina singur, fara **Ctrl+C** , desi robotul de la A nu se opreste niciodata.

M07 — Contor de clickuri (2 stagii)

Butonul „Like” al unei postari e apasat de 4 utilizatori in acelasi timp, fiecare de exact 100.000 de ori. Contorul de like-uri e comun.

Stagiul A: contorul e declarat astfel incat fiecare fir vede imediat ultima valoare scrisa de celelalte — aceeasi garantie ca butonul de la M06, stagiul B — si nimic in plus. Stagiul B: repara contorul fara sa blochezi firele.

Fiecare stagiul afiseaza like-urile numarate, cele asteptate si cate s-au pierdut.

Criteriu de acceptare. Stagiul A pierde like-uri (mai putin de 400.000), desi fiecare fir „vede” valorile celorlalte; programul tipareste singur cate s-au pierdut. Stagiul B numara exact 400.000 la fiecare rulare. In tot fisierul nu apare cuvantul **synchronized** .

M08 — Distribuitor de bonuri

Un automat de bonuri de ordine are 4 usi de intrare. Prin fiecare usa intra 25 de clienti, toti in acelasi timp cu cei de la celelalte usi. Fiecare client primeste un bon cu un numar. Numerele pornesc de la 1 si cresc din 1 in 1. Deasupra ghiseului, un ecran arata numarul ultimului bon dat.

Fiecare usa isi noteaza numerele bonurilor date clientilor ei. La final programul verifica toate cele 100 de bonuri.

Afisezi: primul bon, ultimul bon, ce arata ecranul, cate bonuri sunt dubluri si cate numere lipsesc.

Criteriu de acceptare. Bonurile sunt exact numerele de la 1 la 100, fiecare o singura data: 0 dubluri, 0 lipsa. Primul bon e 1, nu 0. Ecranul arata 100 — nu 99, nu 101. Programul verifica singur fiecare conditie si o marcheaza cand nu se respecta.

M09 — Licitatie cu o singura oferta (2 stagii)

8 ofertanti liciteaza in acelasi timp pe acelasi tablou. Fiecare face 20.000 de oferte crescatoare: oferta numarul **j** are suma **j * 10** plus un numar aleator intre 0 si 9.

Licitatia tine minte cea mai mare oferta si cine a facut-o. O oferta mai mare decat cea curenta trece printr-o **validare** care dureaza (simuleaza cu un mic calcul care consuma timp) si abia apoi devine oferta curenta. O oferta care oricum pierde nu se mai valideaza.

Cand un ofertant isi termina ofertele, incearca sa **adjudece** licitatia: anunta castigatorul. Anuntul dureaza si el: inainte sa inchida licitatia, valideaza inca o data oferta castigatoare. Licitatia se adjudeca **o singura data** — primul care ajunge o inchide, ceilalti o gasesc deja inchisa.

Rulezi 20 de licitatii la rand pentru fiecare stagiul. Dupa fiecare licitatie, programul calculeaza singur cea mai mare oferta reala (fiecare ofertant isi tine propriul maxim) si o compara cu cea retinuta de licitatie. Numara si de cate ori s-a anuntat adjudecarea.

Stagiul A: verifici, apoi scrii — „daca oferta mea e mai mare decat cea curenta, o pun pe a mea”; „daca licitatia nu e inchisa, o inchid si anunt”. Stagiul B: repari ambele, fara **synchronized** si fara lacate.

Criteriu de acceptare. La stagiul A, in majoritatea celor 20 de runde, maximul retinut difera de maximul real sau adjudecarea se anunta de mai multe ori; programul numara singur rundele gresite. La stagiul B, toate cele 20 de runde au maximul corect si exact o adjudecare.

Hinturi

Le citești **dupa** ce te-ai blocat, nu înainte. Niciunul nu-ți da soluția: îți spune unde să cauți și ce întrebare să-ți pui.

M06. Pune `teorie/L02-A1` lângă `teorie/L02-A2` și caută singurul cuvânt care diferă. Dacă la tine robotul de la A **se oprește** totuși: ai în buclă ceva care nu e numărare? Iar ca programul să se termine desigur robotul A nu se oprește niciodată — ai rezolvat deja problema asta la M03.

M07. `teorie/L02-A3` arată de ce garanția de vizibilitate nu ajută la `++`: din câți pași e făcut? Apoi `teorie/L02-B1`.

M08. `teorie/L02-B2`: două metode, aceeași creștere, alt număr întors. Ce valoare are automatul ÎNAINTE de primul client? Ai mai văzut capcana asta la M03, când jurnalul tiparea `0`.

M09. `teorie/L02-B3` arată buclă în care reîncerci până reușești; `teorie/L02-B4` face același lucru pe o referință la obiect, nu pe un număr; `teorie/L02-C1` — „închide-o doar dacă încă e deschisă”. Validarea o faci o dată, înainte de încercare, nu la fiecare reîncercare. Și: ce trebuie să facă buclă ta când încercarea esuează pentru că altcineva a pus între timp o ofertă **mai mare** decât a ta?